

Software Engineering Lab 4 Report

Scenario 21: Checkers

SRN: PES1UG24AM321

Name: Vipin V Lokesh

Date: 6 October 2026

Source repository: github.com/SETAPESU26/21_checkers

Submission repository:

github.com/Vocod3r/Software_engineering_Lab_PES1UG24AM321, folder SE_LAB4_PES1UG24AM321/lab4/21_checkers

1. Overview

The project is a terminal checkers game written in Python using only the standard library. It has four files: main.py (entry point), game.py (turn loop and input), board.py (board set-up and piece movement) and rules.py (move legality and promotion).

The original code let players move and jump, but a jumped piece was never removed, the game never ended, kings could not move backward, captures were optional, and moves gave no feedback. All four tasks in the lab brief were completed. The file structure was kept, no external dependencies were added, and all state stays in memory.

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Vipin>cd C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321>git clone https://github.com/Vocod3r/21_checkers.git
Cloning into '21_checkers'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 15 (delta 2), reused 12 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 7.68 KiB | 1.28 MiB/s, done.
Resolving deltas: 100% (2/2), done.

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321>cd lab4

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4>python main.py
python: can't open file 'C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\main.py': [Errno 2] No such file or directory

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4>dir
Volume in drive C is Windows-SSD
Volume Serial Number is E2A9-0A3D

Directory of C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4

06-10-2026  18:13    <DIR>        .
06-10-2026  18:13    <DIR>        ..
06-10-2026  18:12    <DIR>        21_checkers
               8 File(s)              0 bytes
               3 Dir(s)  153,724,973,056 bytes free

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4>cd 21_checkers

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers>dir
Volume in drive C is Windows-SSD
Volume Serial Number is E2A9-0A3D

Directory of C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers

06-10-2026  18:14    <DIR>        .
06-10-2026  18:14    <DIR>        ..
06-10-2026  18:12    486 board.py
06-10-2026  18:12    1,687 game.py
06-10-2026  18:12         79 main.py
06-10-2026  18:12    3,012 README.md
06-10-2026  18:12    112 requirements.txt
06-10-2026  18:12    840 rules.py
06-10-2026  18:14    <DIR>        __pycache__
               6 File(s)        6,216 bytes
               3 Dir(s)  153,710,051,328 bytes free

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers>cd 21_checkers
The system cannot find the path specified.

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers>python main.py
Checkers - move: sr sc er ec

  0 1 2 3 4 5 6 7
0 . B . B . B .
1 B . B . B .
2 . B . B . B .
3 . . . . .
4 . . . . .
```

Figure 1: Fork cloned into the lab folder and the original game started

2. Defect reproduced in the original code

The original game was run with python main.py and three moves were entered to set up and make a capture:

```
R> 5 0 4 1
B> 2 3 3 2
R> 4 1 2 3
```

Red jumps from row 4, column 1 to row 2, column 3, over the black piece at row 3, column 2. In the last board of Figure 2 that black piece is still on row 3. The captured piece stays on the board and could be captured again. This is also recorded in before.mp4.

```
C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers>python main.py
Checkers - move: sr sc er ec

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 5 0 4 1

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . . .
4 . R . . . . . .
5 . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
B> 2 3 3 2

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . . B . B .
3 . B . . . . .
4 . R . . . . .
5 . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 4 1 2 3

  0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . B . . . . .
4 . . . . . . . .
5 . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
B> q
```

Figure 2: Original code: the jumped black piece remains at row 3, column 2

3. Tasks completed

Task 1: Capture correctness

Problem: In game.py the capture branch only called move_piece, the same as an ordinary move, so the jumped piece was left in place. capture_move in rules.py also compared the jumped piece with the exact string "R" or "B", so a player could jump their own king ("RK").

Fix: Two functions were added to board.py: jumped_square(start, end) returns the square halfway between the start and end of a jump, and remove_piece(board, pos) empties a square and returns the piece that was on it. The capture branch in game.py now moves the piece and then removes the jumped one. In rules.py a new owner(piece) function identifies a piece's side by its first letter, and capture_move uses it.

Result: After 4 1 2 3, row 3, column 2 is empty and the moving piece is on its destination. Jumping your own king is rejected as an invalid move.

```

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers>python main.py
Checkers - move: sr sc er ec (q to quit)

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . .
4 . . . . .
5 . R . R . R .
6 . R . R . R .
7 R . R . R .
R> 5 0 4 1
R moved 5 0 -> 4 1.

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . .
4 . R . . . .
5 . R . R . R .
6 . R . R . R .
7 R . R . R .
B> 2 3 3 2
B moved 2 3 -> 3 2.

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . . . . B
3 . . B . . . .
4 . R . . . .
5 . R . R . R .
6 . R . R . R .
7 R . R . R .
R> 4 1 2 3
R captured B at 3 2 (4 1 -> 2 3).

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . . . . .
4 . . . . .
5 . R . R . R .
6 . R . R . R .
7 R . R . R .
B> q
Game ended.

```

Figure 3: After the Task 1 fix: the jumped piece is removed and the capture is reported

Task 2: Win and no-move detection

Problem: The game loop had no end condition. With no pieces left, the losing side was still asked for a move.

Fix: Four functions were added to rules.py: `in_bounds`, `player_squares` (all squares holding a player's pieces), `piece_can_move` (whether one piece has any legal move or capture) and `has_any_move`. In game.py, the `game_over_message` method checks at the start of each turn whether the current player has no pieces or no legal moves, and the loop prints the result and returns.

Result: The game ends with B has no pieces left. R wins! or B has no legal moves. R wins! and returns to the command prompt.

Task 3: Complete checkers rules

Problem: The direction of movement depended only on the player, so kings moved like ordinary pieces. A player could ignore an available capture, and a turn always ended after one jump.

Fix:

- King movement: `row_directions(piece)` in rules.py returns both directions for a king and one for a man. `simple_move` and `capture_move` use it.
- Forced capture: `captures_from` and `has_capture` were added to rules.py. game.py refuses an ordinary move while a capture is available.
- Multi-capture: after a capture, if the same piece can jump again, game.py stores its square in `self.chain`, keeps the turn with the same player, and only accepts a move from that piece.
- Promotion: red is crowned on row 0 and black on row 7, as in the original. `promote` now returns the squares it crowned so the

game can report it. A piece crowned during a capture ends its turn.

Result: Kings move and capture in both directions, captures are compulsory, and chained jumps are played in one turn.

```
C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LAB4_PES1UG24AM321\lab4\21_checkers>python -c "from game import Checkers; g=Checkers(); g.board=[['.']*8 for _ in range(8)]
j: g.board[0][1]='RK'; g.board[7][0]='B'; g.run()"
Checkers - move: sr sc sr ec (q to quit)

 0 1 2 3 4 5 6 7
0 . . . . . . .
1 . . . . . . .
2 . . . . . . .
3 . . . . . . .
4 . . . . . . .
5 . . . . . . .
6 . . . . . . .
7 B . . . . . .
R> 0 1 1 2
R moved 0 1 -> 1 2.

 0 1 2 3 4 5 6 7
0 . . . . . . .
1 . . RK . . . .
2 . . . . . . .
3 . . . . . . .
4 . . . . . . .
5 . . . . . . .
6 . . . . . . .
7 BK . . . . . .
sr 5 0 4 0
That is not your piece.

 0 1 2 3 4 5 6 7
0 . . . . . . .
1 . . RK . . . .
2 . . . . . . .
3 . . . . . . .
4 . . . . . . .
5 . . . . . . .
6 . . . . . . .
7 BK . . . . . .
sr q
Game ended.
```

Figure 4: A red king moving backward from row 0 to row 1 on a prepared board

Task 4: Move-level feedback

Problem: An accepted move printed only the new board.

Fix: game.py builds one result line for each accepted move and prints it once. A promotion is added to the same line. The rule-checking functions return values and never print, so repeated internal checks do not produce extra output. The board display was also changed to two-character cells so RK and BK stay aligned.

Result: Example output:

```
R moved 5 0 -> 4 1.
R captured B at 3 2 (4 1 -> 2 3).
R moved 1 2 -> 0 1. Promoted to king!
```

```

C:\Users\Vipin\OneDrive\Desktop\SE_Lab\git\Software_engineering_Course_assignments\SE_LABH_PES1UG24AM321\lab4\21_checkers>python main.py
Checkers - move: sr sc er ec (q to quit)

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . .
4 . . . . . . .
5 R . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 5 0 4 1
R moved 5 0 -> 4 1.

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . B . B . B
3 . . . . . . .
4 . R . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 2 3 3 2
R moved 2 3 -> 3 2.

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . . . . . . .
4 . R . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 4 1 2 3
R captured B at 3 2 (4 1 -> 2 3).

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . . . . . . .
4 . . . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> 5 0 4 0
That is not your piece.

 0 1 2 3 4 5 6 7
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . . . . . . .
4 . . . . . . .
5 . . R . R . R .
6 . R . R . R . R
7 R . R . R . R .
R> q
Game ended.

```

Figure 5: Final version: one feedback line per move, aligned board, a rejected selection and a clean quit

4. Testing

The cases in the brief's required-testing list were run against the final code. Positions that are slow to reach in normal play were tested by starting the game from a prepared board.

Case	Input	Observed output	Result
Ordinary move	5 0 4 1	R moved 5 0 -> 4 1.	Pass
Capture	4 1 2 3	R captured B at 3 2 (4 1 -> 2 3). Jumped piece removed.	Pass
Forced capture	5 4 4 5 with a capture available	A capture is available, so you must capture.	Pass
Multiple capture	5 0 3 2 then 3 2 1 4	Another jump is available: go again. Turn passes after the second jump.	Pass
Promotion	1 2 0 1	R moved 1 2 -> 0 1. Promoted to king!	Pass
King moves backward	0 1 1 2	R moved 0 1 -> 1 2.	Pass
King captures backward	2 3 4 5	R captured B at 3 4 (2 3 -> 4 5).	Pass
Own king not capturable	5 4 3 2	Invalid move.	Pass
No pieces left	Capture of last black piece	B has no pieces left. R wins!	Pass
No legal moves	Black piece blocked	B has no legal moves. R wins!	Pass
Illegal move	5 0 4 0	Invalid move.	Pass
Outside board	9 9 9 9	Outside board.	Pass
Non-numeric input	a b c d	Coordinates must be numbers.	Pass
Wrong number of values	1 2	Enter four coordinates.	Pass
Not the player's piece	3 3 4 4	That is not your piece.	Pass
Quit	q	Game ended.	Pass

5. Commit history

Commit	Message	Content
0918bca	Task1 Complete	Checkers code added with the capture fix
71cb64f, 4e18c20	Task2 Completed	Win and no-move detection
06053a3	task3 Completed	King movement, forced and multiple captures
96f24fa	task4 completed	Move feedback and board alignment
8f5e662	Before and After Videos uploaded	before.mp4 and after.mp4

6. Submission items

- Before video: before.mp4 in the submission folder, showing the capture defect.
- After video: after.mp4 in the submission folder, showing the fix and move feedback.
- LLM used: Claude (Anthropic). Chat history link: <https://claude.ai/chat/fdaedb59-1d6e-453e-ac85-6952db638623>

The LLM was used as a coding assistant to inspect the original code, identify defects, and write and explain the changes. The code was run and tested locally against the cases in section 4.